

2026년 스마트해운물류 우수 프로젝트 개 발 보 고 서

2026. 8. 24

프로젝트명	번호	26_HP018
	국문	OceanClaw - 선박 정비 AI Agent
	영문	OceanClaw – AI Agent for Ship Maintenance
작 품 명	OceanClaw	
팀 명	OceanClaw	
공공데이터 활용	☒ 활용 ☐ 미활용	

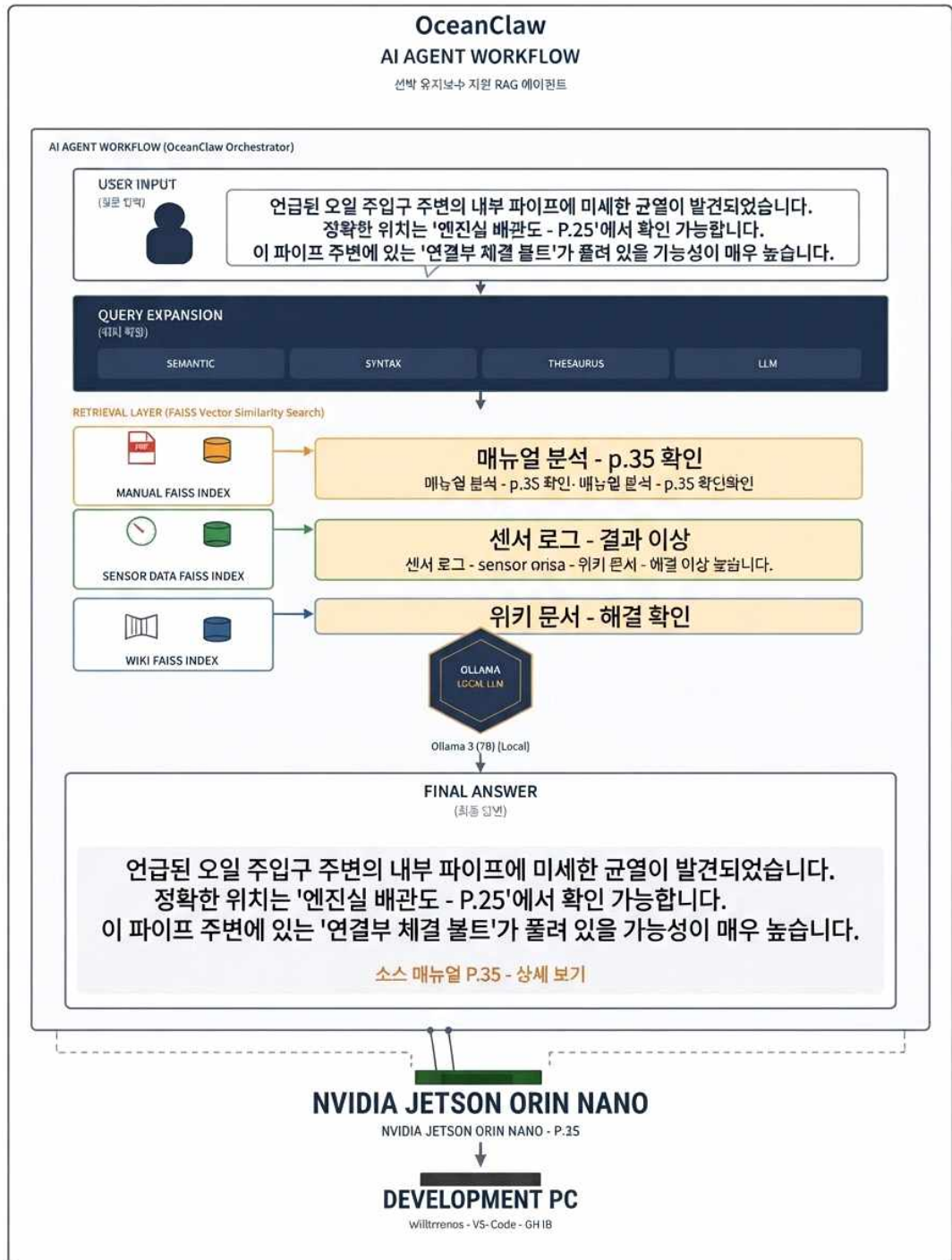
작품 개요

작품 정보		
프로젝트명	국문	OceanClaw - 선박 정비 AI Agent
	작품명	OceanClaw
작품 소개	OceanClaw는 선박 정비 현장의 장비 매뉴얼, 센서 이상 데이터, 정비 지식 문서를 통합 검색하는 로컬 기반 선박 정비 AI Agent이다. 사용자가 자연어로 질문하면 매뉴얼 PDF, 센서 이벤트 CSV, Obsidian Wiki에서 관련 근거를 검색하고, Ollama 기반 로컬 LLM으로 한국어 답변을 생성한다. 답변에는 매뉴얼 페이지와 출처를 함께 제공하여 사용자가 근거를 확인할 수 있도록 하였다.	
작품의 개발배경 및 필요성	선박 정비 업무에서는 장비 매뉴얼, 점검 절차, 센서 이상 이력 등 다양한 정보를 함께 확인해야 한다. 그러나 현장 자료는 PDF, CSV, 문서 파일 등으로 흩어져 있고, 긴 매뉴얼에서 필요한 내용을 빠르게 찾기 어렵다. 또한 선박 환경은 인터넷 연결이 불안정할 수 있어 외부 클라우드 API 의존에 한계가 있다. 이에 본 프로젝트는 매뉴얼과 센서 데이터를 검색 가능한 형태로 변환하고, Jetson Orin Nano에서 로컬 실행 가능한 정비 지원 AI 시스템을 구현하고자 하였다.	
작품의 특징점	OceanClaw의 특징점은 선박 정비에 필요한 매뉴얼, 센서 데이터, 정비 지식을 하나의 AI Agent로 통합한다는 점이다. 사용자는 긴 PDF 매뉴얼을 직접 탐색하지 않고 자연어 질문으로 관련 절차와 근거를 확인할 수 있으며, 답변에는 매뉴얼 페이지와 Wiki 문서 출처가 함께 제공되어 신뢰성을 높였다. 또한 Ollama 기반 로컬 LLM과 Jetson Orin Nano 배포 구조를 적용하여 인터넷 연결이 제한적인 선박 환경에서도 활용 가능하도록 설계하였다. 향후 Gemini, 공공데이터 API, Mattermost, 실제 선박 센서와 연계하여 협업형·현장형 정비 지원 시스템으로 확장할 수 있다.	
작품 기능	OceanClaw는 사용자가 자연어로 정비 질문을 입력하면 선박 매뉴얼 PDF, 센서 이벤트, Obsidian Wiki에서 관련 정보를 검색하고 한국어 답변을 제공한다. 답변에는 출처와 관련 페이지를 함께 표시하여 근거를 확인할 수 있다. 또한 질문과 답변 결과를 Wiki logs에 자동 저장해 정비 지식을 축적할 수 있다. FastAPI 서버와 Mattermost Slash Command를 통해 웹 API와 채팅창에서 모두 사용할 수 있으며, Jetson Orin Nano에서도 실행 가능한 로컬 정비 지원 시스템으로 구성하였다.	
작품의 기대효과 및 활용분야	OceanClaw는 선박 정비 담당자가 긴 매뉴얼을 직접 탐색하지 않고도 필요한 점검 절차와 근거 페이지를 빠르게 확인할 수 있도록 지원한다. 이를 통해 정비 정보 탐색 시간을 줄이고, 센서 이상 상황에서 관련 매뉴얼과 점검 절차를 함께 확인할 수 있다. 또한 로컬 LLM 기반 구조이므로 인터넷 연결이 제한적인 선박 환경에서도 활용 가능하다. 향후 실제 정비 이력 데이터와 울산항만공사 공공데이터 API를 연계하면, 운항 상태를 고려한 점검 시점 안내와 정비 우선순위 판단 기능으로 확장할 수 있다.	

작품 구성도

작품 정보

작품 구성도



본 문

I. 작품 개요

※ 평가항목 : 기획의 적절성

1. 작품 소개

1) 기획의도

선박 정비 현장에서는 장비 매뉴얼, 센서 이상 데이터, 정비 기록 등 다양한 정보를 함께 확인해야 하나, 자료가 PDF, CSV, 문서 파일 등으로 분산되어 있어 필요한 정보를 빠르게 찾기 어렵다.

OceanClaw는 이러한 문제를 해결하기 위해 선박 정비 자료를 통합 검색하고, 근거 기반 답변을 제공하는 로컬 AI Agent로 기획하였다.

인터넷 연결이 제한적인 선박 환경을 고려하여 외부 클라우드 API 의존도를 줄이고,

Ollama 기반 로컬 LLM과 FAISS 검색 구조를 적용하였다.

단순 질의응답 챗봇이 아니라, 매뉴얼 페이지와 센서 이벤트, 정비 Wiki를 함께 참조하여 정비 담당자의 판단을 보조하는 시스템을 목표로 하였다.

2) 작품 정의

OceanClaw는 선박 정비 매뉴얼 PDF, 센서 이벤트 CSV, Obsidian 기반 정비 Wiki를 통합 검색하는 RAG 기반 선박 정비 AI Agent이다.

사용자가 자연어로 질문하면 시스템은 관련 문서와 데이터를 검색하고, 검색된 근거를 기반으로 한국어 정비 답변을 생성한다.

답변에는 가능한 경우 매뉴얼 페이지, 문서명, 검색 결과 등의 출처를 함께 제공하여 사용자가 AI 답변의 근거를 직접 확인할 수 있다.

NVIDIA Jetson Orin Nano Super Developer Kit에서 FastAPI 서버로 실행할 수 있도록 구성하여, 엣지 환경에서의 활용 가능성을 고려하였다.

3) 작품 내용

PDF 매뉴얼 처리

- 선박 엔진 매뉴얼 PDF에서 페이지 단위로 텍스트를 추출한다.
- 추출된 텍스트는 chunk 단위로 분할하여 검색에 적합한 문서 조각으로 변환한다.

Vector Search 기반 검색

- 문서 chunk를 Ollama embedding 모델을 통해 vector로 변환한다.
- 생성된 vector는 FAISS index에 저장하여 사용자 질문과 의미적으로 유사한 문서를 빠르게 검색한다.

센서 데이터 검색

- 원본 CSV 데이터를 정비 이벤트 형식의 sensor_events.csv로 변환한다.
- 센서 이상 이벤트도 embedding하여 매뉴얼과 함께 검색 가능한 데이터로 활용한다.

RAG 기반 답변 생성

- 검색된 매뉴얼 chunk와 센서 이벤트를 context로 구성한다.
- Ollama chat 모델을 이용해 한국어 정비 답변을 생성하며, 답변에는 출처와 관련 페이지를 함께 제공한다.

Obsidian Wiki 연동

- 정비 절차와 지식을 Markdown 문서로 저장할 수 있는 Wiki 구조를 구성하였다.
- Obsidian에서 직접 열람할 수 있으며, 향후 Wiki 문서도 답변 생성에 활용할 수 있도록 검색 index 기반을 마련하였다.

Jetson 배포

- Jetson Orin Nano에서 FastAPI 서버를 실행할 수 있도록 구성하였다.
- 같은 네트워크의 PC에서 Jetson 서버에 접속하여 검색 및 답변 기능을 사용할 수 있도록 하였다.

이후 구현 예정

- Mattermost 실제 채널 연동 완료
 - 현재 구현된 /mattermost/slash endpoint를 Mattermost Slash Command와 완전히 연결한다.
 - 팀원이 Mattermost 채팅창에서 /oceanclaw 명령어로 질문하면 OceanClaw 답변이 채널에 표시되도록 테스트 및 안정화한다.
- 실제 선박 센서 연동
 - 현재는 공개 CSV 데이터를 센서 이벤트로 변환하여 사용하고 있다.
 - 향후 온도, 압력, 진동, RPM 등 실제 센서 데이터를 Jetson에서 수집하고, 이를 정비 이벤트로 변환하여 RAG 검색에 활용할 예정이다.
- 공공데이터 API 연계
 - 울산항만공사 선박 운항정보, 선박제원정보 등 공공데이터 API를 연계할 예정이다.
 - 선박 운항 상태와 제원 정보를 참고하여 점검 시점 안내나 정비 우선순위 판단 기능으로 확장할 수 있다.
- 검색 품질 개선
 - 현재는 Ollama embedding과 FAISS 기반 dense retrieval을 중심으로 검색한다.
 - 향후 BM25 기반 sparse search, hybrid search, reranker 등을 도입하여 부품명, 고장 코드, 전문 용어 검색 성능을 높일 예정이다.
- 사용자 화면 개선
 - 현재는 CLI, FastAPI Swagger UI, Mattermost 연동을 중심으로 테스트하고 있다.
 - 향후 질문 입력창, 답변 화면, 출처 표시, 검색 결과 확인 기능을 포함한 웹 UI를 구현할 예정이다.
- Jetson 운영 안정화
 - Jetson 서버 자동 실행 설정, IP 고정 또는 Tailscale/ngrok 기반 외부 접속 구성을 추가할 예정이다.
 - 이를 통해 팀원이 같은 네트워크가 아니어도 OceanClaw 서버에 접속할 수 있도록 개선한다.
- 검색 평가 자동화
 - 질문 세트와 기대 출처를 정의하고, top-k 검색 결과가 기대 결과와 일치하는지 자동 평가하는 기능을 추가할 예정이다.
 - 이를 통해 검색 정확도와 답변 품질을 정량적으로 확인할 수 있도록 한다.

2. 작품의 개발 배경 및 필요성

1) 작품 제작 동기

선박 정비 업무는 장비 고장이나 이상 상황 발생 시 빠른 판단과 정확한 절차 확인이 중요하다.

그러나 정비 담당자는 긴 PDF 매뉴얼, 센서 로그, 과거 정비 기록을 각각 따로 확인해야 하므로 정보 탐색에 많은 시간이 소요된다.

특히 신규 정비 인력의 경우 매뉴얼 구조와 장비 용어에 익숙하지 않아 필요한 정비 절차를 찾는 데 어려움이 있다.

선박 환경은 인터넷 연결이 항상 안정적이지 않기 때문에, 외부 AI API에 의존하는 시스템은 실제 현장 적용에 제약이 있을 수 있다.

이러한 문제를 해결하기 위해 로컬 환경에서 동작하며, 선박 정비 자료를 통합 검색할 수 있는 AI Agent가 필요하다고 판단하였다.

2) 작품 제작 목적

선박 정비 매뉴얼 PDF에서 필요한 절차와 주의사항을 빠르게 검색할 수 있는 구조를 구현한다.

센서 이상 데이터와 매뉴얼 근거를 함께 활용하여 정비 판단을 보조한다.

AI 답변에 출처와 페이지 정보를 포함하여 사용자가 원문 근거를 검증할 수 있도록 한다.

Ollama 기반 로컬 LLM을 사용하여 API 비용과 인터넷 의존도를 줄인다.

Jetson Orin Nano에서 실행 가능한 구조를 구성하여 엣지 AI 기반 선박 정비 지원 시스템의 가능성을 확인한다.

Obsidian Wiki를 활용하여 정비 지식을 지속적으로 축적하고 검색할 수 있는 기반을 마련한다.

3) 필요성

정비 정보 탐색 시간 단축

- 긴 매뉴얼에서 필요한 내용을 직접 찾는 시간을 줄일 수 있다.

정비 의사결정 보조

- 센서 이상 데이터와 매뉴얼 절차를 함께 확인하여 정비 판단을 지원할 수 있다.

로컬 실행 가능성 확보

- 인터넷이 불안정한 선박 환경에서도 AI 기반 정비 지원 기능을 사용할 수 있다.

지식 축적 체계 마련

- Obsidian Wiki를 통해 정비 절차, 고장 사례, 작업 기록을 체계적으로 저장할 수 있다.

교육 및 실습 활용

- 신규 정비 인력이나 학생들이 선박 정비 절차를 학습하는 데 활용할 수 있다.

3. 작품의 특징 및 장점 / 국내·외 기술 현황 및 본 프로젝트의 차별성

1) 국내·외 기술 현황

국내외 조선·해운 분야에서는 스마트십, 자율운항선박, 선박 상태 모니터링, 예지정비 기술에 대한 연구와 개발이 활발히 진행되고 있다.

선박 엔진, 발전기, 보조기기 등 주요 장비의 센서 데이터를 수집하여 이상 상태를 감지하거나 고장을 예측하는 시스템이 개발되고 있다.

일부 상용 솔루션은 클라우드 기반으로 선박 운항 데이터와 장비 상태 데이터를 분석하여 대시보드 형태로 제공한다.

최근에는 생성형 AI와 RAG 기술을 활용하여 기술 문서, 매뉴얼, 정비 기록을 검색하고 답변하는 AI Assistant가 다양한 산업 분야에서 도입되고 있다.

그러나 선박 정비 분야에서는 매뉴얼 검색, 센서 이벤트 검색, 로컬 LLM, 정비 Wiki를 하나의 구조로 연결한 경량 AI Agent 사례는 아직 제한적이다.

또한 클라우드 기반 AI 서비스는 인터넷 연결, API 비용, 보안 문제로 인해 선박 내부 환경에서 바로 사용하기 어려운 경우가 있다.

2) 기존 유사 기술의 한계

일반 검색 시스템

- 키워드가 정확히 일치해야 검색 품질이 높다.
- 사용자가 장비 용어나 영어 표현을 정확히 알지 못하면 원하는 결과를 찾기 어렵다.

일반 챗봇

- 원본 매뉴얼 근거 없이 답변할 경우 잘못된 정보를 생성할 수 있다.
- 선박 정비처럼 안전과 관련된 분야에서는 출처 없는 답변의 위험성이 크다.

클라우드 기반 AI Assistant

- 인터넷 연결이 필요하다.
- API 사용 비용이 발생한다.
- 선박 내부 자료를 외부 서버로 전송해야 하는 부담이 있다.

단순 센서 모니터링 시스템

- 센서 이상 여부는 확인할 수 있지만, 관련 매뉴얼 절차나 정비 지식과 직접 연결되지는 않는다.

3) 작품의 특징 및 장점

근거 기반 RAG 답변

- PDF 매뉴얼과 센서 이벤트를 먼저 검색한 뒤, 검색 결과를 기반으로 답변을 생성한다.
- 답변에 출처와 페이지 정보를 포함하여 사용자가 원문을 확인할 수 있다.

로컬 LLM 기반 구조

- Ollama를 사용하여 외부 API 없이 로컬에서 LLM을 실행한다.
- API 비용을 줄이고, 인터넷 연결이 불안정한 환경에서도 활용 가능하다.

FAISS 기반 빠른 검색

- 문서 chunk와 센서 이벤트를 embedding vector로 변환하여 FAISS index에 저장한다.
- 사용자의 자연어 질문과 의미적으로 유사한 내용을 빠르게 검색할 수 있다.

한국어 질문과 영어 매뉴얼 검색 보완

- 선박 매뉴얼은 영어 자료가 많으므로 한국어 질문만으로 검색 품질이 낮아질 수 있다.
- 이를 보완하기 위해 query expansion을 적용하여 관련 영어 정비 용어를 함께 검색한다.

Manual/Sensor/Wiki 통합 구조

- 매뉴얼 문서뿐 아니라 센서 이벤트와 Obsidian Wiki 문서까지 검색 대상으로 확장할 수 있다.
- 정비 절차, 센서 이상, 현장 지식을 하나의 AI Agent에서 연결할 수 있다.

Obsidian 기반 지식 축적

- 정비 지식과 작업 기록을 Markdown 문서로 저장한다.
- Git으로 관리하기 쉽고, Obsidian에서 직접 열람 및 수정할 수 있다.

Jetson 기반 엣지 배포

- NVIDIA Jetson Orin Nano에서 FastAPI 서버를 실행하여 같은 네트워크의 PC에서 접속할 수 있도록 구성하였다.
- 선박 내부 또는 실습 환경에서 로컬 서버 형태로 운영할 수 있는 가능성을 확인하였다.

4) 본 프로젝트의 차별성

단순 문서 검색이 아니라 선박 정비 업무 흐름에 맞춘 AI Agent 구조를 구현하였다.

PDF 매뉴얼, 센서 CSV, Wiki 문서를 모두 RAG 검색 대상으로 확장할 수 있도록 설계하였다.

외부 API 기반이 아닌 로컬 LLM 기반 구조를 적용하여 비용과 네트워크 의존도를 줄였다.

Jetson Orin Nano에서 실제 서버 실행과 외부 접속을 확인하여 엣지 AI 적용 가능성을 검증하였다.

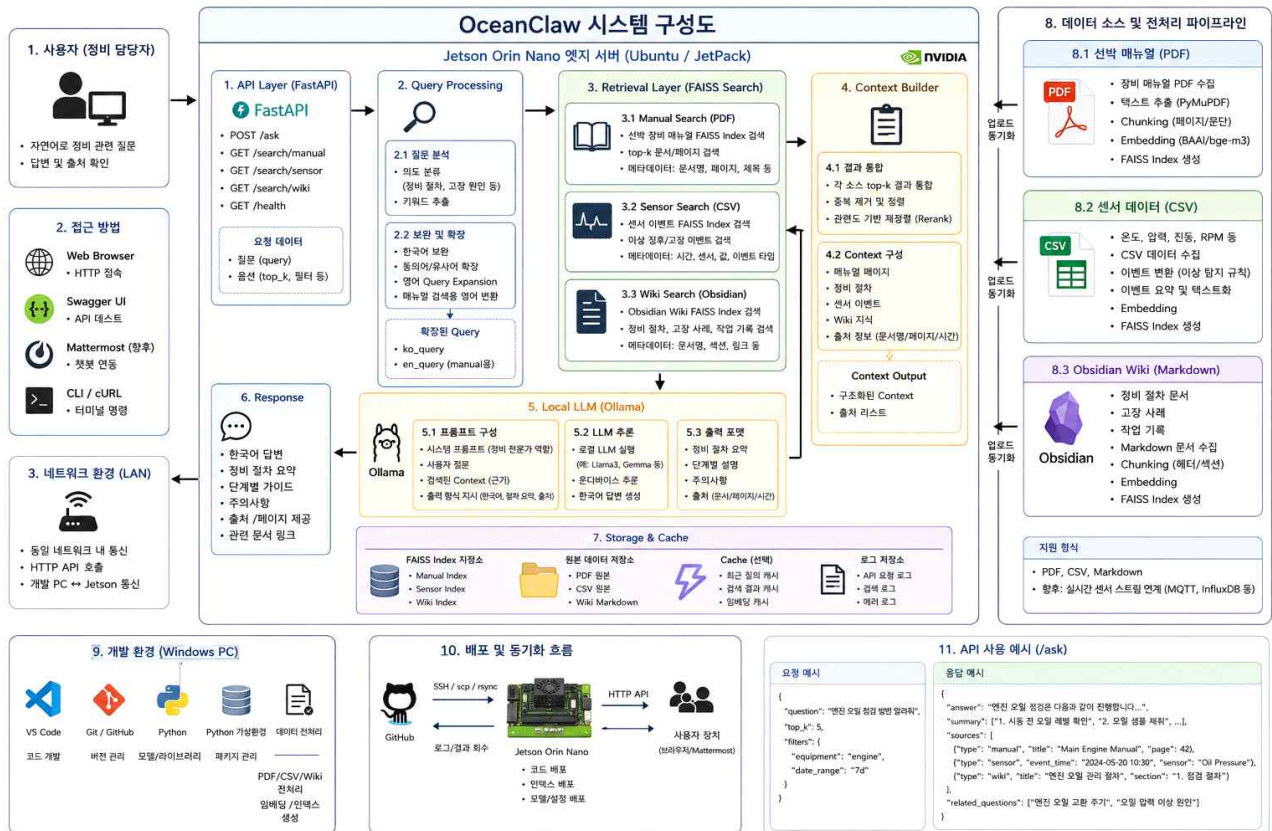
답변 생성뿐 아니라 출처 제공, 검색 결과 확인, 중간 산출물 JSONL 저장 등 검증 가능한 RAG 구조를 구성하였다.

향후 Mattermost 연동과 공공데이터 API 연계를 통해 팀 협업형 선박 정비 지원 시스템으로 확장할 수 있다.

II. 작품 내용

※ 평가항목 : 기획의 적절성, 완성도

1. 목표 시스템 구성도



OceanClaw는 선박 매뉴얼 PDF, 센서 이벤트 CSV, Obsidian Wiki를 통합 검색하여 정비 질문에 답변하는 로컬 AI Agent이다. 사용자의 자연어 질문은 검색용 query로 변환된 뒤 Manual/Sensor/Wiki index에서 관련 근거를 찾고, Ollama 기반 로컬 LLM이 이를 바탕으로 한국어 답변을 생성한다. 답변에는 출처와 관련 페이지를 함께 제공하여 사용자가 근거를 확인할 수 있도록 하였다.

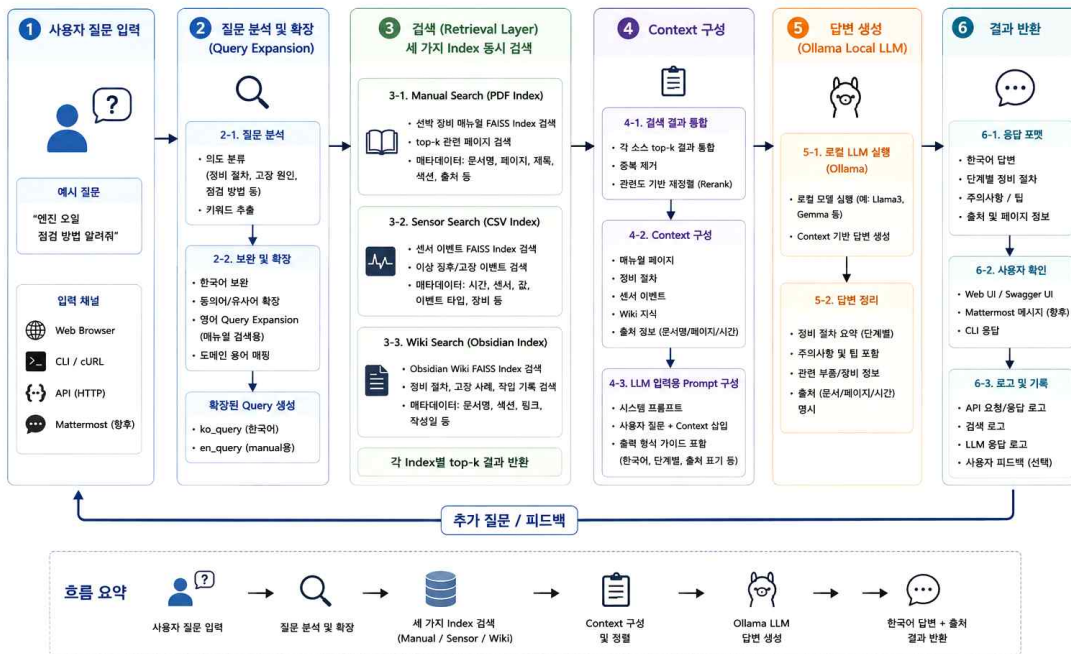
2. 주요 적용 기술

1) RAG 기반 선박 정비 질의응답 기술

본 작품의 핵심 기술은 RAG(Retrieval-Augmented Generation) 구조이다. RAG는 LLM이 자체 지식만으로 답변하지 않고, 외부 문서나 데이터베이스에서 관련 정보를 먼저 검색한 뒤 해당 정보를 바탕으로 답변을 생성하는 방식이다.

선박 정비 분야는 안전과 관련된 정보가 많기 때문에, LLM이 근거 없이 임의로 답변을 생성하는 방식은 위험할 수 있다. 따라서 OceanClaw는 매뉴얼 PDF와 센서 이벤트 CSV를 먼저 검색하고, 검색된 근거를

OceanClaw 서비스 흐름도 (정비 AI Agent 흐름)



context로 구성한 뒤 답변을 생성하도록 설계하였다. Obsidian Wiki 문서도 향후 답변 context에 포함할 수 있도록 검색 기반을 마련하였다.

질문 입력

- > 관련 문서 검색
- > 검색 결과 추출
- > LLM context 구성
- > 답변 생성
- > 출처 제공

이를 통해 사용자는 AI 답변뿐 아니라 답변의 근거가 되는 문서명과 페이지도 함께 확인할 수 있다.

2) PDF 텍스트 추출 및 문서 전처리

선박 장비 매뉴얼은 대부분 PDF 형태로 제공되므로, RAG 시스템에 활용하기 위해 먼저 텍스트 추출이 필요하다. OceanClaw는 PDF를 page 단위로 읽고 다음과 같은 구조로 저장한다.

```
{
  "source": "yanmar_6lf_operation_manual.pdf",
  "page": 69,
  "text": "Check Oil Level in Engine ..."
}
```

page 단위로 저장하는 이유는 답변 생성 후 사용자가 원본 매뉴얼의 특정 페이지를 확인할 수 있도록 하기 위해서이다. 또한 추출 결과를 JSONL 형태로 저장하여 텍스트가 정상적으로 추출되었는지 사람이 직접 검토할 수 있도록 하였다.

3) Chunk 분할 기술

PDF 한 페이지 또는 전체 문서를 그대로 embedding하면 검색 단위가 너무 커져 정확도가 떨어질 수 있다. 따라서 OceanClaw는 문서를 일정한 크기의 chunk로 나누어 검색 단위로 사용하였다.

Chunk 분할에는 LangChain의 RecursiveCharacterTextSplitter를 활용하였다. 이 방식은 단순 글자 수 기준으로 자르는 것보다 문단, 줄바꿈, 문장 경계를 우선 고려할 수 있다는 장점이 있다.

PDF page text

-> chunk 1

-> chunk 2

-> chunk 3

또한 chunk overlap을 적용하여 문서가 잘리는 경계에서 중요한 문맥이 사라지지 않도록 하였다.

4) Embedding 및 Vector Search

Embedding은 텍스트의 의미를 숫자 벡터로 변환하는 기술이다. 사용자의 질문과 문서 chunk를 같은 vector 공간에 배치하면, 의미적으로 비슷한 문서를 검색할 수 있다.

OceanClaw는 Ollama embedding 모델을 사용하여 문서 chunk와 질문을 vector로 변환하고, 변환된 vector를 FAISS index에 저장하였다.

문서 chunk

-> Ollama Embedding

-> Vector

-> FAISS Index

사용자가 질문하면 질문도 embedding으로 변환하고, FAISS에서 가장 유사한 chunk를 top-k 방식으로 검색한다. FAISS는 대규모 vector 검색에 적합한 라이브러리로, 로컬 환경에서 빠른 검색이 가능하다는 장점이 있다.

5) Query Expansion

실제 선박 매뉴얼은 영어로 작성된 경우가 많지만, 사용자는 한국어로 질문할 가능성이 높다. 초기 검색 테스트에서 “엔진 오일 점검 방법”이라는 한국어 질문은 정확한 페이지를 찾지 못했지만, “engine oil level check dipstick MIN MAX”와 같은 영어 표현을 포함하면 정확도가 크게 향상되었다.

이를 해결하기 위해 OceanClaw는 query expansion을 적용하였다.

입력 질문:

엔진 오일 점검 방법 알려줘

확장 질문:

엔진 오일 점검 방법 알려줘

engine oil level dipstick MIN MAX check inspect procedure

이 방식은 한국어 질문과 영어 매뉴얼 사이의 표현 차이를 보완하여 검색 정확도를 높인다.

6) Ollama 기반 Local LLM

OceanClaw는 답변 생성을 위해 Ollama 기반 로컬 LLM을 사용하였다. 초기 계획서에는 Google Gemini 사용이 포함되어 있었지만, 현재 개발 단계에서는 비용과 인터넷 의존성을 줄이고 Jetson 기반 로컬 실행 가능성을 검증하기 위해 Ollama를 우선 적용하였다.

Ollama를 사용하면 개발 PC 또는 Jetson에서 모델을 직접 실행할 수 있으며, 외부 API 호출 없이 답변 생성이 가능하다. 또한 시스템 구조는 LLM 모듈을 분리하여 설계하였기 때문에, 향후 필요에 따라 Gemini와 같은 외부 LLM API로 교체하거나 병행 사용할 수 있다.

주요 장점은 다음과 같다.

- API 비용 절감
- 인터넷 의존도 감소
- 선박 내부망 또는 폐쇄망 환경 적용 가능
- Jetson 기반 엣지 AI 실험 가능

LLM prompt에는 검색된 context만 사용하도록 조건을 넣어 환각 가능성을 줄이고자 하였다.

7) Sensor Event 변환 및 검색 기술

원본 센서 CSV 데이터는 수치 중심이기 때문에 바로 자연어 검색에 사용하기 어렵다. 따라서 OceanClaw는 CSV row를 정비 이벤트 설명 형태의 text로 변환하였다.

예: 엔진 온도 상승, 압력 저하, 이상 상태 발생 가능성 ...

변환된 text를 embedding하고 sensor FAISS index를 생성하여 센서 이상 관련 질문도 검색할 수 있도록 하였다. 이를 통해 단순 매뉴얼 검색뿐 아니라 센서 이상 데이터와 정비 절차를 연결할 수 있는 기반을 마련하였다.

8) FastAPI 기반 서버 기술

CLI만으로는 팀원이 사용하기 어렵기 때문에 FastAPI 서버를 구현하였다. 주요 API는 다음과 같다.

GET /health

POST /ask

GET /search/manual

GET /search/sensor

Jetson에서 서버를 0.0.0.0:8000으로 실행하면, 같은 네트워크의 데스크탑 PC에서 API 문서와 질의응답 기능을 사용할 수 있다. 이를 통해 OceanClaw를 단순 로컬 스크립트가 아니라 네트워크 기반 서비스 형태로 확장하였다.

9) Obsidian Wiki 기반 지식 관리

Obsidian은 Markdown 기반 지식 관리 도구이다. OceanClaw는 wiki/폴더를 Obsidian Vault처럼 구성하여 정비 절차, 장비 구성요소, 작업 로그를 저장할 수 있도록 하였다.

wiki/

components/

procedures/

logs/

templates/

Wiki 문서는 사람이 직접 읽고 수정할 수 있으며, 향후 AI 검색 대상으로도 활용할 수 있다. 이를 통해 일회성 답변을 넘어 정비 지식을 지속적으로 축적하는 구조를 마련하였다.

10) Jetson Orin Nano 기반 Edge AI 배포

본 작품은 NVIDIA Jetson Orin Nano Super Developer Kit에서 실행할 수 있도록 배포 환경을 구성하였다. Jetson에 Python 환경, Ollama, FAISS index, FastAPI 서버를 구성하고 같은 네트워크의 PC에서 접속하는 방식으로 테스트하였다.

이를 통해 클라우드 서버 없이도 현장 장비에서 AI Agent를 실행할 수 있는 가능성을 확인하였다.

3. 작품 개발 환경

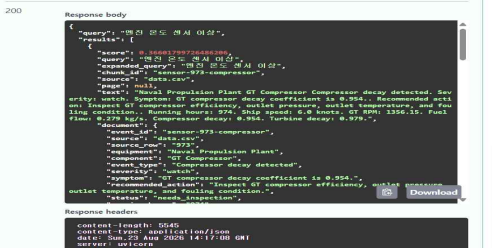
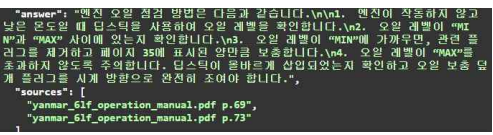
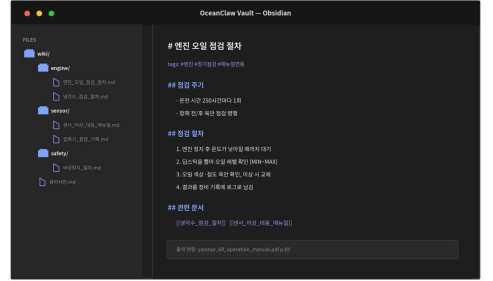
구분		상세내용
SW 개발환경	OS	Windows 11 개발 PC, Ubuntu 기반 NVIDIA Jetson Orin Nano 환경
	개발환경(IDE)	Visual Studio Code, PowerShell, Jetson Terminal
	개발도구	Python 가상환경, Git, GitHub, Ollama, FastAPI, FAISS, Obsidian
	개발언어	Python
	기타사항	PDF RAG 구현을 위해 pypdf, langchain-text-splitters, faiss-cpu, numpy, fastapi, uvicorn등을 사용하였다. 로컬 LLM 실행을 위해 Ollama를 사용하였고, embedding 및 chat 모델을 로컬에서 호출하도록 구성하였다.
HW 구성장비	디바이스	NVIDIA Jetson Orin Nano Super Developer Kit, Windows 개발 데스크탑 PC
	센서	실제 선박 센서는 직접 연결하지 않았으며, 공개 데이터셋 기반 CSV 데이터를 선박 센서 이벤트 형식으로 변환하여 사용하였다. 향후 실제 선박 엔진 센서, 온도 센서, 압력 센서, 진동 센서 등과 연계 가능하다.
	통신	동일 네트워크 내 Wi-Fi/LAN 기반 통신, SSH 접속, FastAPI HTTP 통신
	언어	Python 기반 서버 및 데이터 처리 로직 구현
	기타사항	Jetson에서 FastAPI 서버를 0.0.0.0:8000으로 실행하고, 데스크탑 PC에서 http://<Jetson-IP>:8000/docs로 접속하여 API 테스트를 진행하였다.
프로젝트 관리환경	형상관리	Git, GitHub
	의사소통관리	팀원 간 역할 분담을 통해 데이터 수집과 개발을 병행하였으며, 개발 진행 상황은 문서화 및 GitHub 업로드를 통해 공유하였다. 향후 Mattermost 연동을 통해 팀 협업 환경에서 AI Agent를 사용할 수 있도록 확장할 예정이다.
	기타사항	프로젝트 산출물 관리를 위해 data/, index/, scripts/, oceanclaw/, wiki/, docs/폴더로 구분하였다. 실행에 필요한 데이터와 index는 GitHub에 포함하고, .env, .venv, 캐시 파일 등은 제외하였다.

4. 작품 구성 기능

1) 주요 기능 목록

구분	기능	설명	현재진척도(%)
SW	선박 매뉴얼 RAG 검색	선박 엔진 매뉴얼 PDF에서 텍스트를 추출하고 chunk 분할, embedding, FAISS index 생성을 통해 사용자의 자연어 질문과 관련된 매뉴얼 내용을 검색한다.	100
	센서 이벤트 검색	원본 CSV 데이터를 정비 이벤트 형식으로 변환하고, 센서 이상 정보도 embedding하여 검색할 수 있도록 구현하였다.	100
	로컬 LLM 답변 생성	Ollama 기반 로컬 LLM을 사용하여 검색된 매뉴얼 및 센서 데이터를 바탕으로 한국어 정비 답변을 생성하고 출력을 함께 제공한다.	100
	FastAPI 서버	/ask, /health, /search/manual, /search/sensorAPI를 제공하여 같은 네트워크의 PC에서 Jetson 서버에 접속해 기능을 테스트할 수 있도록 구현하였다.	100
	Obsidian Wiki 기반 지식 관리	정비 절차와 지식을 Markdown 문서로 저장하고 Obsidian에서 열람할 수 있는 Wiki 구조를 구성하였다. Wiki 검색 index도 일부 구현하였다.	90
	Mattermost 및 공공데이터 연계	팀 협업 채널에서 질문할 수 있는 Mattermost 연동과 울산항만공사 공공데이터 API 연계는 향후 확장 기능으로 계획 중이다. 완성 가능시점: 8/28	0
HW	Jetson Orin Nano 배포 환경	구매HW. NVIDIA Jetson Orin Nano Super Developer Kit에 프로젝트를 배포하고 FastAPI 서버 실행 및 외부 PC 접속을 확인하였다.	100
	NVMe SSD 및 주변 장비	구매HW. Jetson 실행 환경 구성을 위해 256GB NVMe SSD, USB 설치 장치, DP to HDMI 케이블 등을 사용하였다.	100
	실제 선박 센서 연계	실제 선박 센서는 아직 직접 연결하지 않았으며, 현재는 공개 데이터셋 기반 CSV를 센서 이벤트로 변환하여 대체하였다. 향후 실제 센서 확보 시 연계 가능하다.	0

2) SW 개발 기능 상세내용

기능	설명	작품실물사진
선박 매뉴얼 RAG 검색	선박 엔진 매뉴얼 PDF에서 텍스트를 추출하고, chunk 분할 후 Ollama embedding과 FAISS index를 이용하여 자연어 질문과 관련된 매뉴얼 내용을 검색하도록 구현하였다. 검색 결과에는 문서명, 페이지, chunk ID, 유사도 점수, 원문 내용이 포함된다.	
센서 이벤트 검색	원본 CSV 데이터를 OceanClaw용 센서 이벤트 형식으로 변환하고, 각 이벤트 설명을 embedding하여 sensor FAISS index를 생성하였다. 이를 통해 센서 이상, 고장 징후, 점검 이벤트와 관련된 질문을 검색할 수 있다.	
로컬 LLM 답변 생성	검색된 매뉴얼 및 센서 데이터를 context로 구성하고 Ollama 기반 로컬 LLM을 호출하여 한국어 답변을 생성한다. 답변에는 사용자가 원문 근거를 확인할 수 있도록 출처와 페이지 정보를 함께 제공한다.	
FastAPI 서버	/health, /ask, /search/manual, /search/sensorendpoint를 제공하는 FastAPI 서버를 구현하였다. Jetson에서 서버를 실행하고 같은 네트워크의 데스크탑 PC에서 http://<Jetson-IP>:8000/docs로 접속하여 API 테스트를 진행하였다.	
Obsidian Wiki 기반 지식 관리	wiki/폴더를 Obsidian Vault처럼 사용할 수 있도록 구성하고, 정비 절차와 지식을 Markdown 문서로 저장할 수 있도록 하였다. Wiki 문서도 검색 index로 확장할 수 있도록 chunk 및 FAISS index 생성 구조를 구현하였다.	
Mattermost 및 공공데이터 연계	현재는 향후 확장 기능으로 계획 중이다. Mattermost는 팀 채팅 환경에서 OceanClaw에 질문하고 답변을 받을 수 있도록 연동할 예정이며, 공공데이터는 울산항만공사 선박 운항정보 또는 선박제원정보 API를 참조하는 방식으로 확장할 계획이다.	

- SW 주요 목표대비 현재 개발정도 상세 설명

현재 SW 개발은 핵심 RAG 기능을 중심으로 대부분 구현된 상태이다. 선박 매뉴얼 PDF를 page 단위로

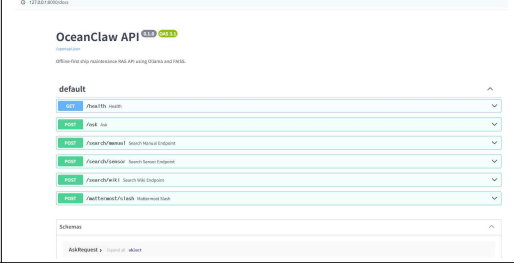
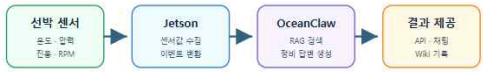
추출하고, LangChain 기반 chunk 분할을 수행한 뒤 Ollama embedding과 FAISS를 이용하여 검색 index를 생성하였다. 사용자의 자연어 질문에 대해 관련 매뉴얼 chunk를 검색하고, 검색된 내용을 context로 구성하여 Ollama LLM이 한국어 답변을 생성하는 구조까지 구현하였다.

센서 데이터의 경우 실제 선박 센서와 직접 연결하지는 않았지만, 공개 데이터셋 기반 CSV를 OceanClaw용 센서 이벤트 데이터로 변환하고 검색 가능한 형태로 구성하였다. 이를 통해 매뉴얼 검색뿐 아니라 센서 이상 이벤트 검색까지 가능한 구조를 마련하였다.

또한 FastAPI 서버를 구현하여 CLI 실행에 그치지 않고 외부 PC에서 API 형태로 기능을 테스트할 수 있도록 하였다. Jetson Orin Nano에서 서버를 실행하고 데스크탑 PC에서 /docs와 /ask요청을 확인함으로써, 로컬 네트워크 기반 AI 정비 지원 시스템의 동작을 검증하였다.

Obsidian Wiki는 정비 지식을 Markdown 문서로 축적하기 위한 구조를 구성하였으며, Wiki 문서 검색 index도 일부 구현하였다. 다만 Mattermost 연동과 울산항만공사 공공데이터 API 연계는 아직 구현 전이며, 향후 확장 기능으로 개발할 예정이다.

3) HW 개발 기능 상세내용

기능/부품	설명	작품실물사진
Jetson Orin Nano 배포 환경	NVIDIA Jetson Orin Nano Super Developer Kit를 구매하여 OceanClaw 실행 장비로 사용하였다. Jetson에 Python 환경, Ollama, 프로젝트 코드를 구성하고 FastAPI 서버 실행을 확인하였다.	
NVMe SSD 및 저장장치	Jetson 실행 환경 구성을 위해 256GB NVMe SSD를 사용하였다. 프로젝트 코드, Python 환경, Ollama 모델, index 파일 등을 저장하기 위한 장치로 활용하였다.	
네트워크 통신 환경	Jetson과 데스크탑 PC를 동일 네트워크에 연결하고 SSH 및 HTTP 기반 FastAPI 통신을 확인하였다. 데스크탑에서 Jetson IP로 접속하여 API 문서와 질문 응답 기능을 테스트하였다.	
실제 선박 센서 연계	현재 실제 선박 센서는 직접 연결하지 않았으며, 공개 데이터셋 기반 CSV를 센서 이벤트 데이터로 변환하여 대체하였다. 향후 온도, 압력, 진동 등 실제 센서 데이터를 연결하면 실시간 정비 진단 기능으로 확장할 수 있다.	향후 실제 선박 센서 연동 구조  실제 센서 데이터를 Jetson에서 수집하고, OceanClaw가 매뉴얼 센서 Wiki 문서를 활용 검색하여 정비 방안을 지원

HW 주요 목표대비 현재 개발정도 상세 설명

HW 개발은 Jetson Orin Nano 기반 엡지 배포 환경 구축을 중심으로 진행하였다. 초기에는 Jetson OS 설치, USB 설치 장치 인식, 모니터 출력, DP to HDMI 케이블 호환성, NVMe SSD 장착 등 환경 구성에서 여러 문제가 발생하였다. 그러나 문제를 단계별로 해결하면서 Jetson 부팅과 서버 실행 환경을 구성하였다.

현재 Jetson에서는 OceanClaw 프로젝트를 실행할 수 있으며, FastAPI 서버를 0.0.0.0:8000으로 실행하여 같은 네트워크의 데스크탑 PC에서 접속할 수 있는 상태까지 확인하였다. 이를 통해 OceanClaw가 단순 개발 PC용 프로그램이 아니라 엡지 장비에서 동작 가능한 로컬 AI Agent 구조임을 검증하였다.

다만 실제 선박 센서와의 직접 연계는 아직 진행하지 않았다. 현재는 공개 데이터셋 기반 CSV를 센서 이벤트로 변환하여 검색 대상으로 사용하고 있으며, 향후 실제 센서 장비가 확보되면 Jetson의 GPIO, USB, Serial, MQTT 등의 방식으로 센서 데이터를 수집하고 OceanClaw의 sensor RAG 구조와 연결할 수 있다.

5. 기타 사항 [본문에서 표현되지 못한 작품의 가치(Value)] 및 제작 노력

본 작품은 단순히 LLM API를 호출하는 챗봇이 아니라, 선박 정비 현장에서 실제로 사용할 수 있는 근거 기반 RAG 시스템을 목표로 개발하였다.

- 사용자의 질문에 바로 답변을 생성하는 방식이 아니라, 먼저 매뉴얼 PDF, 센서 이벤트 CSV, Wiki 문서를 검색한 뒤 검색 결과를 기반으로 답변하도록 구성하였다.
- 이를 통해 LLM의 환각 가능성을 줄이고, 사용자가 원본 매뉴얼의 출처와 페이지를 직접 확인할 수 있도록 하였다.
- 정비 분야는 안전과 직결되기 때문에 단순한 자연어 답변보다 “근거를 확인할 수 있는 답변”이 중요하다고 판단하였다.

데이터 처리 과정을 검증 가능하게 설계한 점에 가치가 있다.

- PDF에서 텍스트를 추출한 결과를 바로 embedding하지 않고, page 단위 JSONL로 저장하여 추출 품질을 확인할 수 있도록 하였다.
- 이후 chunk 결과도 별도 JSONL로 저장하여 문서가 어떤 단위로 나뉘었는지 확인할 수 있게 하였다.
- 검색 결과에는 score, source, page, chunk_id, text를 포함하여 검색 품질을 사람이 직접 검토할 수 있도록 하였다.
- 이러한 구조는 RAG 시스템에서 답변 오류가 발생했을 때 원인을 단계별로 추적할 수 있다는 장점이 있다.

로컬 LLM 기반 구조를 적용하여 비용과 네트워크 의존성을 줄였다.

- 초기 계획서에는 Gemini 사용이 포함되어 있었지만, API 비용과 인터넷 연결 문제를 고려하여 Ollama 기반 로컬 LLM 구조로 변경하였다.
- 선박 환경은 인터넷 연결이 항상 안정적이지 않을 수 있으므로, 로컬에서 실행 가능한 구조는 현장 적용 가능성을 높인다.
- 또한 개발 과정에서 반복 테스트를 수행할 때 외부 API 비용이 발생하지 않는다는 장점이 있다.

Jetson Orin Nano에서 동작 가능한 엣지 AI 구조를 구현한 점이 차별적이다.

- 많은 AI 시스템은 개발 PC나 클라우드 서버에서만 동작하지만, 본 작품은 NVIDIA Jetson Orin Nano Super Developer Kit에서 FastAPI 서버를 실행하고 외부 PC에서 접속하는 단계까지 검증하였다.
- 이를 통해 선박 내부 네트워크, 실습실, 항만 현장 등 제한된 환경에서도 운용 가능한 AI Agent 구조를 실험하였다.
- Jetson 환경에서는 응답 속도와 timeout 문제가 발생했으나, timeout 설정을 조정하고 서버 실행 구조를 정리하여 실제 동작 가능한 상태로 개선하였다.

한국어 질문과 영어 매뉴얼 사이의 검색 문제를 해결하기 위해 query expansion을 적용하였다.

- 선박 장비 매뉴얼은 영어로 작성된 경우가 많지만, 국내 사용자는 한국어로 질문할 가능성이 높다.
- 초기 검색 테스트에서 한국어 질문만으로는 정확한 매뉴얼 페이지가 검색되지 않는 문제가 있었다.
- 이를 해결하기 위해 “엔진 오일 점검”과 같은 한국어 표현에 “engine oil level, dipstick, MIN, MAX, check” 등의 영어 정비 용어를 함께 추가하는 query expansion을 구현하였다.
- 이는 단순 번역이 아니라 실제 검색 결과를 검토하며 필요한 키워드를 보완한 개선 과정이라는 점에서 실용적 가치가 있다.

센서 데이터와 매뉴얼 검색을 함께 고려한 구조를 만들었다.

- 일반 문서 검색 시스템은 매뉴얼 검색에 그치는 경우가 많지만, 본 작품은 센서 CSV 데이터를 정비 이벤트 형태로 변환하여 검색 대상으로 포함하였다.
- 이를 통해 장비 이상 상태와 매뉴얼 점검 절차를 연결할 수 있는 기반을 마련하였다.
- 실제 선박 센서가 연결되면 온도, 압력, 진동, RPM 등 다양한 장비 상태 데이터를 바탕으로 정비 의사결정을 지원할 수 있다.

Obsidian Wiki 기반 지식 축적 구조를 설계하였다.

- 정비 지식은 한 번의 답변으로 끝나는 것이 아니라, 작업 기록과 고장 사례가 계속 축적되어야 한다.
- 본 작품은 wiki/폴더를 Obsidian Vault처럼 구성하여 정비 절차, 부품 정보, 작업 로그를 Markdown 문서로 저장할 수 있도록 하였다.
- Markdown 기반 구조는 Git으로 관리하기 쉽고, 사람이 직접 읽고 수정할 수 있으며, AI 검색 대상으로도 활용할 수 있다.
- 이를 통해 AI Agent와 사람의 정비 지식 관리가 연결될 수 있는 기반을 마련하였다.

알고리즘 구성 측면에서 단계별 RAG 파이프라인을 직접 구현하며 이해도를 높였다.

- PDF 추출, chunk 분할, embedding 생성, FAISS index 생성, 검색, prompt 구성, 답변 생성 단계를 직접 구성하였다.
- LangChain은 전체 프레임워크로 무리하게 사용하지 않고, chunk 분할처럼 필요한 부분에만 선택적으로 도입하였다.
- 이를 통해 프로젝트 규모에 맞는 단순하고 검증 가능한 구조를 유지하였다.

보안 및 운영 측면에서도 의미가 있다.

- 외부 API에 선박 매뉴얼이나 정비 데이터를 전송하지 않고 로컬에서 처리할 수 있기 때문에, 민감한 정비 자료를 외부로 보내지 않는 구조로 확장할 수 있다.
- .env, .venv, 캐시 파일 등은 GitHub에 올리지 않도록 관리하고, 실행에 필요한 데이터와 index 파일은 팀원이 쉽게 실행할 수 있도록 포함하는 방식으로 정리하였다.
- 향후 실제 운영 시에는 내부망 배포, 인증, 접근 제어 등을 추가하여 보안성을 높일 수 있다.

제작 과정에서 많은 시행착오를 겪으며 실제 완성도를 높였다.

- PDF 추출 구조 재설계, chunk 방식 변경, embedding 검색 품질 확인, query expansion 적용, sensor CSV 변환, API 서버 구현, Jetson 배포 등 여러 단계를 반복적으로 검증하였다.
- 특히 Jetson 환경 구성 과정에서 USB 설치, 모니터 출력, SSH 접속, 서버 포트 충돌, timeout

문제 등을 직접 해결하였다.

- 이러한 과정은 단순 코드 작성이 아니라 실제 장비에서 동작하는 시스템을 만들기 위한 제작 노력으로 볼 수 있다.

6. 향후 개선사항

Manual, Sensor, Wiki 통합 답변 고도화

- 현재는 매뉴얼 검색과 센서 검색, Wiki 검색의 기반 구조가 구현되어 있다.
- 향후에는 질문 하나에 대해 manual, sensor, wiki를 동시에 검색하고, 가장 적절한 근거를 조합하여 답변하는 통합 Agent 구조로 개선할 예정이다.

Hybrid Search 도입

- 현재는 Ollama embedding과 FAISS 기반 dense retrieval을 중심으로 검색한다.
- 향후 BM25 기반 sparse retrieval을 추가하여 정확한 부품명, 고장 코드, 전문 용어 검색 성능을 높일 계획이다.
- Dense search와 sparse search를 결합한 hybrid search를 적용하면 의미 기반 검색과 키워드 기반 검색의 장점을 함께 활용할 수 있다.

Reranker 적용

- 현재는 FAISS 유사도 점수 기준으로 top-k 결과를 사용한다.
- 향후 reranker를 추가하여 검색된 후보 문서 중 실제 질문과 가장 관련성이 높은 문서를 다시 정렬할 계획이다.
- 이를 통해 답변 품질과 출처 정확도를 높일 수 있다.

Mattermost 연동

- 계획서의 협업 요구사항을 반영하기 위해 Mattermost와 연동할 예정이다.
- 팀원이 Mattermost 채널에서 정비 질문을 입력하면 OceanClaw가 FastAPI /ask를 호출하고, 답변과 출처를 채널에 반환하는 구조로 개발할 수 있다.
- 이를 통해 별도 웹 화면 없이도 팀 협업 환경에서 AI Agent를 사용할 수 있다.

Obsidian 자동 기록 기능

- 현재는 Obsidian Wiki 구조와 검색 index 기반을 구현하였다.
- 향후 사용자의 질문, AI 답변, 참고 출처, 정비 메모를 자동으로 Markdown 문서에 기록하는 기능을 추가할 예정이다.
- 이를 통해 반복되는 정비 질문과 답변이 팀의 지식 자산으로 축적될 수 있다.

울산항만공사 공공데이터 API 연계

- 향후 공공데이터포털의 울산항만공사 선박 운항정보, 선박제원정보 등을 연계할 계획이다.
- 선박 입출항 상태, 접안 상태, 선박 제원 등을 참고하여 정비 시점 안내나 점검 우선순위 판단 기능으로 확장할 수 있다.
- 실제 API를 코드에 연결하면 개발보고서에서 공공데이터 활용 프로젝트로 명시할 수 있다.

실제 선박 센서 데이터 연계

- 현재는 공개 CSV 데이터를 센서 이벤트 형태로 변환하여 사용하였다.
- 향후 실제 온도, 압력, 진동, RPM 등 선박 장비 센서를 연결하면 실시간 이상 감지와 정비 추천 기능으로 확장할 수 있다.
- Jetson의 GPIO, USB, Serial, MQTT 등의 통신 방식을 활용할 수 있다.

Jetson 운영 안정화

- 현재 Jetson에서 서버 실행과 외부 접속은 확인하였다.
- 향후 systemd 서비스를 설정하여 Jetson 부팅 시 OceanClaw API 서버가 자동 실행되도록 개선할 예정이다.
- 또한 IP 고정 또는 공유기 DHCP 예약을 설정하여 팀원들이 항상 동일한 주소로 접속할 수 있도록 할 계획이다.

사용자 인터페이스 개선

- 현재는 FastAPI Swagger UI 또는 CLI 중심으로 테스트하고 있다.
- 향후 간단한 웹 UI를 만들어 사용자가 질문을 입력하고 답변, 출처, 검색 결과를 한 화면에서 확인할 수 있도록 개선할 예정이다.
- 정비 담당자가 기술적인 명령어를 몰라도 사용할 수 있는 형태로 발전시키는 것이 목표이다.

평가 자동화

- 검색 질문 세트와 기대 출처를 만들어 top-k 검색 결과가 올바른지 자동으로 평가하는 기능을 추가할 예정이다.
- 답변 품질 평가 기준도 만들어 검색 정확도, 출처 일치율, 응답 시간 등을 기록할 수 있도록 개선할 계획이다.

데이터 확장 및 전문성 강화

- 현재는 제한된 공개 매뉴얼과 CSV 데이터를 기반으로 개발하였다.
- 향후 다양한 선박 장비 매뉴얼, 실제 정비 이력, 고장 코드, 부품 정보 등을 추가하여 전문성을 높일 예정이다.
- 데이터가 축적될수록 OceanClaw는 특정 장비를 넘어 범용 선박 정비 AI Agent로 확장될 수 있다.

III. 프로젝트 수행 내용

※ 평가항목 : 완성도, 협업의 적절성

1. 프로젝트 수행일정

프로젝트 기간		2026. 06. 20. ~ 2026. 10. 10.				
구분	추진내용	프로젝트 기간				
		6월	7월	8월	9월	10월
계획	프로젝트 주제 선정 및 요구사항 분석	계획	완료			
	선박 정비 AI Agent 전체 구조 기획	계획		완료		
	Gemini, LangChain, Obsidian, Mattermost, Jetson등 계획서 요구사항 검토	계획		완료		
분석	선박 정비 업무 흐름 및 RAG 적용 가능성 분석	계획		완료		
	PDF 매뉴얼, CSV 센서 데이터 Wiki 문서 활용 방식 분석	계획		완료		
	Gemini API와 Ollama 로컬 LLM 비교 및 기술 선택	계획		완료		
설계	PDF 추출, chunk, embedding, FAISS 검색 파이프라인 설계		진행	완료		
	Manual/Sensor/Wiki 통합 RAG 구조 설계		진행	완료		
	FastAPI 서버 및 Jetson 배포 구조 설계		진행	완료		
개발	PDF 텍스트 추출 및 JSONL 저장 기능 구현		진행	완료		
	LangChain 기반 chunk 분할 기능 구현		진행	완료		
	Ollama embedding 및 FAISS index 생성 기능 구현		진행	완료		
	Manual 검색 및 query expansion 기능 구현		진행	완료		
	Ollama 기반 RAG 답변 생성 기능 구현		진행	완료		
	Sensor CSV 변환 및 sensor FAISS 검색 기능 구현		진행	완료		
	FastAPI API 서버 구현		진행	완료		
	Obsidian Wiki 구조 및 Wiki 검색 index 구현			진행		
	Mattermost 연동 및 공공데이터 API 연계 개발			진행		
	웹 UI 또는 협업용 인터페이스 개선				계획	
테스트	검색 결과 top-k 검증 및 답변 품질 확인			진행		
	Jetson Orin Nano 배포 및 외부 접속 테스트			진행		
	실제 데이터 추가 후 검색/답변 평가			계획		
종료	최종 기능 정리 및 GitHub 산출물 관리				계획	
	개발보고서, 발표자료, 시연 준비				계획	
	최종 결과물 제출				계획	

2. 팀원의 세부목표 수립 및 협업을 위한 노력

팀원	역할	프로젝트 세부목표
서한녕	기획 및 프로젝트관리(PM)	프로젝트 전체 목표 설정, 수행계획 관리, 일정 조율, 멘토 피드백 반영, 개발보고서 및 발표자료 정리, 팀원별 작업 범위 조정
이규원	데이터 수집 및 전처리	선박 정비 매뉴얼 PDF, 센서 데이터, 공공데이터 후보 조사 및 수집, 데이터 출처 정리, 평가 질문 및 기대 출처 정리
박건우	AI/RAG 백엔드 개발	PDF 텍스트 추출, chunk 분할, embedding, FAISS 검색, query expansion, Ollama 기반 답변 생성, FastAPI API 서버 구현
김현준	Jetson 배포 및 협업 기능 개발	Jetson Orin Nano 실행 환경 구성, 서버 배포 및 외부 접속 테스트, Obsidian Wiki 구조 관리, 향후 Mattermost 및 공공데이터 API 연동 지원

1) 프로젝트 수행(협업) 방안

- 팀원별 역할을 기획, 프론트엔드, 백엔드, 선행기술 분석으로 나누어 병렬적으로 프로젝트를 수행하였다.
- 프로젝트 초기에는 수행계획서와 개요서를 바탕으로 전체 목표 기능을 정리하고, 핵심 기능과 확장 기능을 구분하였다.
 - 핵심 기능: 선박 매뉴얼 RAG 검색, 센서 이벤트 검색, 로컬 LLM 답변 생성, FastAPI 서버
 - 확장 기능: Obsidian Wiki, Mattermost 연동, 공공데이터 API 연계, Jetson 운영 안정화

협업 과정에서는 팀원 간 진행 상황을 공유하며, 데이터 수집과 개발을 분리하여 진행하였다.

- 데이터 담당 팀원은 선박 매뉴얼, 센서 데이터, 공공데이터 후보를 조사하였다.
- 개발 담당 팀원은 데이터가 준비되면 바로 처리할 수 있도록 PDF 추출, CSV 변환, FAISS index 생성 구조를 먼저 구현하였다.

GitHub를 활용하여 코드와 산출물을 관리하였다.

- 코드 변경 이력은 Git으로 관리하고, 실행에 필요한 파일과 제외해야 할 파일을 .gitignore로 구분하였다.
- Jetson에서 git pull후 실행할 수 있도록 프로젝트 구조와 실행 방법을 정리하였다.

문서화 작업을 병행하였다.

- README, 프로젝트 구조 문서, 개발보고서 초안, Obsidian Wiki 문서를 작성하여 팀원이 프로젝트 상태를 이해할 수 있도록 하였다.
- 개발 중 발생한 문제점과 해결 방법을 기록하여 보고서와 발표자료 작성에 활용하였다.

기능 개발은 단계별로 검증하며 진행하였다.

- PDF 추출 결과 확인
- chunk 결과 확인
- 검색 top-k 결과 확인
- Ollama 답변 생성 확인
- FastAPI 서버 실행 확인
- Jetson 외부 접속 확인

향후 협업 계획은 다음과 같다.

- 프론트엔드 팀원은 사용자 질의응답 화면과 결과 표시 UI를 개발한다.
- 백엔드 팀원은 Mattermost 연동, 공공데이터 API 연계, 검색 품질 평가 기능을 추가한

- 다.
- 전체 팀원은 실제 데이터 기반 테스트 질문을 만들고, 검색 결과와 답변 품질을 함께 검토한다.

3. 프로젝트 추진 과정에서의 문제점(애로사항) 및 해결방안

1) 프로젝트 관리 측면

초기 요구사항 범위가 넓어 우선순위 설정에 어려움이 있었다.

- 계획서에는 Gemini, LangChain, Mattermost, Obsidian, LLM Wiki, Jetson 배포 등 다양한 요구사항이 포함되어 있었다.
- 처음에는 모든 기능을 동시에 구현하려다 보니 어떤 기능부터 개발해야 하는지 명확하지 않았다.
- 이를 해결하기 위해 먼저 핵심 기능을 “PDF 매뉴얼 검색 → 답변 생성 → 센서 데이터 검색 → API 서버 → Wiki/Jetson 확장” 순서로 나누어 단계별로 개발하였다.
- 결과적으로 RAG의 기본 흐름을 먼저 완성한 뒤 부가 기능을 확장하는 방식으로 프로젝트를 관리할 수 있었다.

실제 데이터 확보가 예상보다 어려웠다.

- 선박 정비 이력, 장비 고장 로그, 센서 이상 데이터는 공개적으로 구하기 어려웠다.
- 초반에는 임시 PDF와 CSV를 만들어 파이프라인을 먼저 검증하였고, 이후 공개 선박 엔진 매뉴얼과 외부 공개 데이터셋을 활용하여 테스트를 진행하였다.
- 팀원이 데이터 수집을 맡고, 개발 담당자는 데이터가 들어왔을 때 바로 처리할 수 있도록 PDF 추출, CSV 변환, index 생성 구조를 먼저 구현하였다.

개발 방향이 중간에 변경되었다.

- 초기 계획서에는 Google Gemini 사용이 포함되어 있었지만, API 비용과 인터넷 의존성 문제가 있었다.
- 특히 Jetson Orin Nano에서 로컬 실행을 고려해야 했기 때문에 Gemini 중심 구조는 실제 배포 목적과 맞지 않는 부분이 있었다.
- 이를 해결하기 위해 Ollama 기반 로컬 LLM 구조로 변경하였다.
- 계획서와 완전히 다른 방향으로 보이지 않도록 LangChain은 chunk 분할에 일부 도입하고, 전체 구조는 로컬 RAG Agent로 재정의하였다.

파일 구조와 산출물이 점점 복잡해졌다.

- PDF 추출, chunk 생성, embedding, FAISS index, sensor 변환, Wiki 생성 등 기능이 늘어나면서 파일이 많아졌다.
- 어느 파일이 어떤 역할을 하는지 헷갈리는 문제가 발생하였다.
- 이를 해결하기 위해 scripts, oceanclaw 모듈, data, index, wiki, docs 등으로 역할을 나누고, 중간 산출물을 JSONL/CSV/FAISS 형태로 구분하였다.
- 또한 README와 문서 파일을 작성하여 프로젝트 구조를 정리하였다.

GitHub 업로드 범위 결정에 어려움이 있었다.

- 일반적으로 data와 index 파일은 Git에서 제외하는 경우가 많지만, Jetson에서 git pull만으로 실행 가능한 상태를 만들고 싶었다.
- 반대로 .env, .venv, 캐시 파일 등은 보안 및 용량 문제로 업로드하면 안 되었다.
- 최종적으로 보안 위험이 큰 파일은 제외하고, 실행에 필요한 data와 index 파일은 포함하는 방식으로 .gitignore를 조정하였다.

Jetson 환경 설정에 많은 시간이 소요되었다.

- OS 설치, USB 문제, 모니터 연결, NVMe SSD 장착, SSH 접속, IP 확인 등 개발 외적인 환경 설정에서 예상보다 많은 시간이 걸렸다.
- 특히 설치 USB 용량, DP to HDMI 케이블, 화면 출력 문제 등으로 진행이 지연되었다.
- 문제를 하나씩 분리하여 확인하고, 최종적으로 Jetson에서 서버를 실행하고 데스크탑에서 접속하는 단계까지 완료하였다.

2) 작품 개발 측면

PDF 텍스트 추출 결과를 바로 신뢰하기 어려웠다.

- PDF는 파일마다 구조가 다르기 때문에 텍스트가 정상적으로 추출되지 않거나, 페이지 순서와 문장 흐름이 어긋날 수 있었다.
- 이를 해결하기 위해 PDF 추출 결과를 JSONL로 저장하고, page, source, text를 직접 확인할 수 있도록 하였다.
- 이후 chunk 결과도 별도 JSONL로 저장하여 검색이 이상할 때 어느 단계에서 문제가 발생했는지 추적할 수 있도록 하였다.

Chunk 분할 방식 선택이 어려웠다.

- 처음에는 단순 글자 수 기준으로 자르는 방법도 고려했지만, 정비 매뉴얼은 절차와 주의사항이 문맥 단위로 이어지는 경우가 많았다.
- 너무 작게 자르면 문맥이 부족하고, 너무 크게 자르면 검색 정확도가 떨어질 수 있었다.
- 이를 해결하기 위해 LangChain의 RecursiveCharacterTextSplitter를 사용하여 문단, 줄바꿈, 문장 경계를 고려하는 방식으로 chunk를 나누었다.
- chunk size와 overlap을 조정할 수 있도록 하여 추후 검색 품질 개선이 가능하게 하였다.

한국어 질문과 영어 매뉴얼 사이의 검색 품질 문제가 있었다.

- 사용자는 한국어로 질문하지만, 실제 매뉴얼은 영어로 작성되어 있었다.
- 처음에 "엔진 오일 점검 방법"이라고 검색했을 때 관련 없는 페이지가 검색되었고, "engine oil level check dipstick MIN MAX"처럼 영어 키워드를 넣었을 때 정확한 결과가 나왔다.
- 이를 해결하기 위해 query expansion 기능을 구현하였다.
- 한국어 정비 질문에 관련 영어 키워드를 추가하여 영어 매뉴얼에서도 검색 품질이 향상되도록

하였다.

Dense retrieval만으로 모든 질문을 해결하기에는 한계가 있었다.

- Ollama embedding과 FAISS를 사용한 dense search는 의미 기반 검색에 장점이 있지만, 정확한 부품명이나 고유명사 검색에서는 sparse search가 더 유리할 수 있다.
- 현재는 1차 구현을 위해 dense retrieval을 선택하였고, 향후 BM25 기반 sparse search와 결합한 hybrid search로 개선할 계획을 세웠다.

답변 생성에서 환각 가능성을 줄여야 했다.

- LLM은 검색 결과에 없는 내용도 그럴듯하게 생성할 수 있다.
- 정비 분야에서는 잘못된 답변이 안전 문제로 이어질 수 있기 때문에, prompt에서 검색된 context만 사용하도록 제한하였다.
- 또한 답변에 출처와 페이지 정보를 함께 제공하여 사용자가 원문을 검증할 수 있도록 하였다.

Sensor CSV 데이터를 RAG에 맞게 변환하는 과정이 필요했다.

- 원본 CSV는 수치형 데이터 중심이라 바로 자연어 검색에 사용하기 어려웠다.
- 각 row를 정비 이벤트 설명 형태의 text로 변환하고, 이를 embedding하여 sensor FAISS index를 생성하였다.
- 이를 통해 센서 이상 데이터도 문서처럼 검색할 수 있게 되었다.

Manual 검색과 Sensor 검색을 어떻게 통합할지 고민이 있었다.

- 처음에는 키워드 기반 router를 만들었지만, 사용자가 항상 정해진 키워드로 질문하지는 않는다는 문제가 있었다.
- 이를 해결하기 위해 route를 강제로 하나만 선택하기보다 manual과 sensor를 모두 검색하고, 필요에 따라 검색 점수와 route 옵션을 활용하는 방식으로 개선하였다.

Jetson에서 LLM 응답 timeout 문제가 발생하였다.

- 데스크탑에서는 Ollama 응답이 빠르게 끝났지만, Jetson에서는 모델 응답 시간이 길어 Python 요청이 timeout되는 문제가 있었다.
- Ollama 자체는 동작했지만 Python 코드에서 timeout 값이 짧아 실패한 것이 원인이었다.
- 이를 해결하기 위해 OLLAMA_TIMEOUT기본값을 60초에서 300초로 늘렸다.
- 또한 Jetson에서 서버 실행 시 포트 충돌 여부와 기존 서버 실행 상태를 확인하는 방법을 정리하였다.

API 서버 실행 중 패키지 누락 문제가 발생하였다.

- Jetson에서 FastAPI 서버를 실행할 때 uvicorn모듈이 없어 오류가 발생하였다.
- 이를 해결하기 위해 requirements에 필요한 패키지를 명확히 포함하고, Jetson에서 패키지를 설치한 뒤 실행하도록 정리하였다.

Obsidian 연동의 의미를 구체화하는 데 시간이 필요했다.

- 처음에는 Obsidian이 답변 생성과 직접 연결되는지, 단순 문서 저장소인지 역할이 명확하지 않았다.
- 최종적으로 Obsidian은 정비 지식을 저장하는 Markdown 기반 Wiki로 정의하였다.
- Wiki 문서 생성, Wiki 구조 구성, Wiki 검색 index 생성까지 구현하여 향후 답변 자동 기록 기

능으로 확장할 수 있는 기반을 마련하였다.

4. 프로젝트를 통해 배우거나 느낀 점

이번 프로젝트를 통해 AI Agent 개발은 단순히 LLM API를 호출하는 것이 아니라, 데이터를 어떻게 준비하고 검색하며, 어떤 근거를 바탕으로 답변을 생성할지 설계하는 과정이라는 것을 배웠다. 특히 RAG 시스템에서는 PDF 추출, chunk 분할, embedding, 검색, prompt 구성 중 어느 단계라도 부정확하면 최종 답변 품질이 떨어질 수 있다는 점을 경험하였다.

처음에는 Gemini, LangChain, Jetson, Obsidian 등의 기술을 개별적으로만 이해하고 있었다. 개발을 진행하면서 Ollama/Gemini는 답변 생성, LangChain은 문서 분할, FAISS는 검색 index, Obsidian은 지식 축적 공간, Jetson은 엣지 배포 환경이라는 식으로 각 기술의 역할과 연결 구조를 이해하게 되었다.

로컬 LLM의 장점과 한계도 함께 경험하였다. Ollama를 사용하면 API 비용과 인터넷 의존도를 줄일 수 있었지만, Jetson 환경에서는 응답 속도, timeout, 모델 크기 선택과 같은 현실적인 문제가 발생하였다. 이를 통해 실제 현장 적용에는 모델 성능뿐 아니라 하드웨어 성능과 운영 환경까지 함께 고려해야 한다는 것을 느꼈다.

데이터의 중요성도 크게 느꼈다. 코드가 정상적으로 동작해도 적절한 매뉴얼과 센서 데이터가 없으면 좋은 답변을 만들기 어렵다. 특히 선박 정비 데이터는 공개적으로 구하기 쉽지 않아, 데이터 수집과 정제, 평가 질문 설계가 프로젝트 품질에 큰 영향을 준다는 것을 알게 되었다.

문제를 단계별로 검증하는 습관의 중요성을 배웠다. 답변이 이상할 때 바로 LLM 문제로 판단하지 않고, PDF 추출 결과, chunk 결과, 검색 결과, prompt를 차례대로 확인해야 했다. JSONL, CSV 같은 중간 산출물을 저장해둔 덕분에 문제 원인을 추적할 수 있었다.

Jetson 배포를 통해 실제 장비에서 시스템을 실행하는 과정도 중요한 개발 경험이라는 것을 느꼈다. USB 설치, 모니터 연결, SSH 접속, IP 확인, 패키지 설치, 서버 실행 등 운영 환경을 직접 구성하면서 로컬 PC에서 코드가 동작하는 것과 실제 장비에서 서비스로 실행하는 것은 다르다는 점을 배웠다.

최종적으로 OceanClaw를 개발하면서 스마트해운물류 분야에서 AI가 실질적으로 활용될 수 있다는 가능성을 확인하였다. 선박 정비는 전문 용어와 절차가 많고 자료가 복잡하기 때문에, 근거 기반 검색과 답변 생성이 현장 업무를 보조하는 데 도움이 될 수 있다고 느꼈다. 향후 실제 선박 정비 데이터와 공공 데이터 API를 연계한다면 더 실용적인 현장형 정비 지원 시스템으로 발전시킬 수 있을 것이다.

IV. 작품의 기대효과 및 활용분야

※ 평가항목 : 결과의 활용성

1. 작품의 활용(적용)분야

선박 정비 현장

- 선박 엔진, 발전기, 보조기기 등 주요 장비의 점검 절차 확인에 활용할 수 있다.
- 정비 담당자가 긴 매뉴얼을 직접 탐색하지 않고 필요한 절차와 관련 페이지를 빠르게 확인할 수 있다.

선박 관리 및 해운사 운영 지원

- 장비 이상 상황, 센서 이벤트, 정비 기록을 함께 확인하는 정비 지원 도구로 활용할 수 있다.
- 선박별 정비 이력과 장비 매뉴얼을 연결하여 유지보수 관리 효율을 높일 수 있다.

정비 교육 및 실습

- 신규 정비 인력, 학생, 실습생이 선박 장비 매뉴얼과 점검 절차를 학습하는 교육용 AI 도구로 활용할 수 있다.
- 답변과 함께 출처를 제공하므로 학습자가 원문 근거를 확인하며 이해할 수 있다.

엣지 AI 기반 현장 시스템

- Jetson Orin Nano 기반 배포를 통해 로컬 환경에서 실행 가능한 정비 지원 시스템으로 활용할 수 있다.
- 향후 자동 실행, IP 고정, 외부 접속 설정을 보완하면 선박 내부 네트워크 또는 제한된 네트워크 환경에서도 운용 가능하다.

정비 지식 관리 시스템

- Obsidian Wiki를 활용하여 정비 절차, 고장 사례, 질문/답변 기록을 Markdown 문서로 축적할 수 있다.
- 팀 단위로 정비 지식을 공유하고 검색하는 내부 지식 관리 도구로 활용할 수 있다.

향후 항만·해운물류 서비스 확장

- 울산항만공사 공공데이터 API와 연계하면 선박 운항 상태, 입출항 정보, 선박 제원 등을 참고한 정비 지원 서비스로 확장할 수 있다.

2. 작품의 활용에 의한 기대효과(사용자)

정비 정보 탐색 시간 단축

- 사용자는 긴 PDF 매뉴얼을 직접 처음부터 탐색하지 않고, 자연어 질문으로 필요한 절차와 관련 페이지를 빠르게 찾을 수 있다.
- 정비 현장에서 필요한 정보를 찾는 시간을 줄여 작업 효율을 높일 수 있다.

정비 판단 보조

- 센서 이상 데이터와 매뉴얼 근거를 함께 검색하여 장비 상태를 판단하는 데 도움을 줄 수 있다.
- 단순 센서값 확인을 넘어 관련 점검 절차와 주의사항까지 함께 확인할 수 있다.

답변 신뢰성 향상

- AI 답변에 매뉴얼 출처와 페이지 정보를 함께 제공하여 사용자가 원문을 직접 검증할 수 있다.
- 출처 없는 일반 챗봇 답변보다 정비 업무에 더 안전하게 활용할 수 있다.

신규 인력 교육 지원

- 장비 용어나 매뉴얼 구조에 익숙하지 않은 신규 정비 인력도 질문을 통해 필요한 정보를 쉽게 찾을 수 있다.
- 정비 절차, 안전 주의사항, 센서 이상 상황을 학습하는 데 도움을 줄 수 있다.

인터넷 의존도 감소

- Ollama 기반 로컬 LLM과 FAISS 검색을 사용하므로 외부 API 연결 없이도 주요 기능을 사용할 수 있다.
- 선박 내부, 항만 현장, 실습실 등 네트워크가 제한적인 환경에서도 활용 가능성이 있다.

정비 지식 축적

- Obsidian Wiki를 통해 정비 절차와 작업 기록을 지속적으로 저장할 수 있다.
- 시간이 지날수록 조직 내부의 정비 지식이 축적되어 반복 문제 대응과 인수인계에 도움이 된다.

협업 효율 향상

- FastAPI 서버를 통해 같은 네트워크의 팀원이 시스템에 접속하여 검색과 답변 기능을 사용할 수 있다.
- 향후 Mattermost와 연동하면 팀 채팅 환경에서 정비 질문과 답변을 공유할 수 있다.

3. 작품의 기대가치

기존 서비스와의 차별성

- 기존 매뉴얼 검색은 키워드 기반이 많아 사용자가 정확한 용어를 알아야 원하는 정보를 찾을 수 있다.
- OceanClaw는 embedding 기반 의미 검색을 사용하여 자연어 질문과 관련된 매뉴얼 내용을 검색할 수 있다.
- 일반 챗봇과 달리 검색된 문서 근거를 기반으로 답변하며, 출처와 페이지 정보를 함께 제공한다.
- 단순 센서 모니터링 시스템과 달리 센서 이상 이벤트를 매뉴얼 절차와 연결할 수 있는 구조를 가진다.
- Obsidian Wiki를 통해 현장 정비 지식을 지속적으로 축적하고 검색 대상으로 확장할 수 있다.

가격 우위성

- Gemini, GPT 등 외부 API 기반 시스템은 사용량에 따라 지속적인 비용이 발생할 수 있다.
- OceanClaw는 Ollama 기반 로컬 LLM을 사용하여 반복적인 질의응답 테스트와 현장 운용 시 API 비용 부담을 줄일 수 있다.

- FAISS, FastAPI, Obsidian, Ollama 등 오픈소스 및 무료 도구를 중심으로 구성하여 초기 도입 비용을 낮출 수 있다.
- Jetson Orin Nano와 같은 소형 엣지 장비에서 운용 가능하므로 고가의 서버 인프라 없이 현장 적용을 실험할 수 있다.

사회적 가치

- 선박 정비 절차를 빠르게 확인할 수 있어 작업자의 실수 가능성을 줄이고 안전한 정비 환경 조성에 기여할 수 있다.
- 신규 정비 인력의 학습 부담을 줄이고, 숙련자의 정비 지식을 문서화하여 조직 내 기술 전수에 도움을 줄 수 있다.
- 해운·항만 분야의 디지털 전환과 스마트 정비 체계 구축에 기여할 수 있다.

경제적 가치

- 정비 정보 탐색 시간을 줄여 업무 효율을 높이고, 장비 이상 대응 시간을 단축할 수 있다.
- 반복되는 고장 사례와 점검 기록을 축적하면 예방정비와 정비 계획 수립에 활용할 수 있다.
- 향후 실제 선박 정비 이력, 센서 로그, 공공데이터 API와 연계하면 선박 관리자, 해운사, 항만 운영기관을 대상으로 한 정비 지원 솔루션으로 확장 가능하다.

확장 가능성

- Mattermost 연동을 통해 팀 채팅 기반 정비 AI Assistant로 확장할 수 있다.
- 울산항만공사 공공데이터 API를 연계하여 선박 운항 상태와 항만 정보를 함께 고려하는 정비 지원 기능으로 발전시킬 수 있다.
- Hybrid Search, reranker, 평가 자동화 기능을 추가하여 검색 정확도와 답변 품질을 지속적으로 개선할 수 있다.
- 다양한 선박 장비 매뉴얼과 실제 정비 데이터를 추가하면 특정 장비에 한정되지 않는 범용 선박 정비 AI Agent로 발전할 수 있다.